Tenemos el siguiente caso de Estudio:

Caso de Negocio: Sistema de Gestión para Laboratorio de Análisis Clínicos
El laboratorio clínico recibe diariamente a pacientes con órdenes médicas para la realización de estudios. Actualmente, gran parte del proceso es manual y se desea informatizar para mejorar la trazabilidad de las muestras, reducir errores y agilizar la entrega de resultados.
Objetivo: Automatizar el circuito completo desde el ingreso del paciente con una orden médica, hasta la entrega del resultado.

1. Ingreso del Paciente
   - El paciente se presenta con una orden médica.
   - La recepcionista registra los datos personales y los estudios requeridos.
   - La recepcionista le informa la fecha para retirar el resultado y le imprime la información que es el usuario y clave asignada para ingresar al sitio web
2. Generación de análisis clínico
   - En el paso previo, la recepcionista registra los estudios requeridos por parte del medico, cada pedido/analisis puede incluir uno o varios estudios específicos (ej. hemograma, glucosa, colesterol, etc.) según lo que solicita el médico.
   - El análisis clínico es diferente para cada paciente, es decir varía según la combinación de estudios solicitados.
     NOTA: Se pide implementar una solución flexible para construir análisis clínicos compuestos a partir de estudios individuales, utilizando un patrón de diseño apropiado para facilitar la extensión y mantenimiento del sistema.
     Explicar que categoría y patron GOF usan y por qué
3. Identificación de muestras
   - El sistema genera un código QR vinculado a los estudios y al paciente.
   - Se imprime una etiqueta con el QR y se adhiere al tubo de muestra.
4. Extracción de Sangre
   - El bioquímico escanea el QR, verifica los estudios y realiza la extracción.
5. Análisis de Laboratorio
   - El bioquímico vuelve a escanear el QR en el laboratorio.
   - El sistema indica los estudios solicitados por el médico y los reactivos necesarios para llevar a cabo dicho análisis.
   - Se cargan los valores de cada análisis, y cuando se carga el ultimo valor se finaliza el estudio.
   - Algunos análisis, se cargan en forma automática, es decir que tiene una solución que lee el valor del reactivo y lo graba. Los otros reactivos que los procesos de lectura NO están automatizados y el bioquímico, debe colocar el valor en forma manual al sistema.
6. Entrega de Resultados
   - El sistema genera un informe en PDF.
   - El informe se envía por correo electrónico al paciente, también el paciente puede ingresar al sistema web del laboratorio para ver todos los estudios históricos realizados, el paciente ingresa con usuario y clave dado por el laboratorio

Ejemplo de análisis del laboratorio

Prueba	Proceso	Reactivo utilizado	Tpo de espera Minutos	Acción luego del tiempo
Colesterol total	automatizado	Reactivo enzimático CT	10	Sistema toma el valor
HDL (colesterol bueno)	automatizado	Reactivo HDL selectivo	12	Sistema toma el valor
LDL (calculado o directo)	automatizado	Reactivo LDL directo	15	Sistema toma el valor
Triglicéridos	MANUAL	Reactivo GPO-POD	35	-

Con este caso vamos a comenzar a modelar nuestra solución.

Recordemos que el modelo es el TODO. Las vistas son las formas de mirar ese todo desde distintos ángulos.

*Nuestro **modelo completo** incluirá: casos de uso o tarjeta de usuario o especificación, clases o servicios o microservicios, diagrama se actividad, secuencias, base de datos(DER), reglas de negocio, etc.etc.*

Como nuestro proceso es UP, donde pone foco en el paradigma OO y quiero representar un modelo desde diferentes vistas, puedo usar:

- *La **vista funcional**: qué hace el sistema a través de Casos de uso.*
- *La **vista estática**: cómo se organiza el código a través del diagrama de Clases*
- *La **vista dinámica**: cómo interactúan los objetos a través del diagramas de iteracción (comunicación, secuencia)*

Vamos a comenzar desde la Vista funcional, podemos usar diagrama de CU, y diag de actividad

***Diagrama de Casos de Uso,** representa **las funcionalidades (casos de uso)** que ofrece el sistema desde el punto de vista del usuario o actor externo.*

Vamos a definir los Actores del Caso de negocio:

- *Paciente, persona que se presenta con una orden médica, accede a resultados, etc.*
- *Recepcionista, persona que registra al paciente, los estudios requeridos y entrega información.*
- *Bioquímico, persona que realiza extracciones, analiza muestras, carga resultados*
- *Sistema de lectura automática, carga automáticamente valores de ciertos estudios*

Vamos a definir CU:

Es recomendable leer el caso de negocio y marcar las funcionalidades que pueden llegar a ser CU, luego las evaluaremos para ver si corresponden

Caso de Negocio: Sistema de Gestión para Laboratorio de Análisis Clínicos
El laboratorio clínico recibe diariamente a pacientes con órdenes médicas para la realización de estudios. Actualmente, gran parte del proceso es manual y se desea informatizar para mejorar la trazabilidad de las muestras, reducir errores y agilizar la entrega de resultados.
Objetivo: Automatizar el circuito completo desde el ingreso del paciente con una orden médica, hasta la entrega del resultado.

1. Ingreso del Paciente
   - El paciente se presenta con una orden médica.
   - La recepcionista registra los datos personales y los estudios requeridos.
   - La recepcionista le informa la fecha para retirar el resultado y le imprime la información que es el usuario y clave asignada para ingresar al sitio web

   → Registrar ingreso de paciente. Registrar estudios solicitados

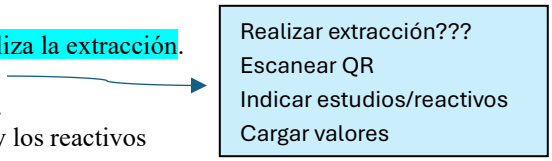
2. Generación de análisis clínico
   - En el paso previo, la recepcionista registra los estudios requeridos por parte del médico, cada pedido/análisis puede incluir uno o varios estudios específicos (ej. hemograma, glucosa, colesterol, etc.) según lo que solicita el médico.
   - El análisis clínico es diferente para cada paciente, es decir varía según la combinación de estudios solicitados.
     NOTA: Se pide implementar una solución flexible para construir análisis clínicos compuestos a partir de estudios individuales, utilizando un patrón de diseño apropiado para facilitar la extensión y mantenimiento del sistema.
     Explicar que categoría y patrón GOF usan y por qué

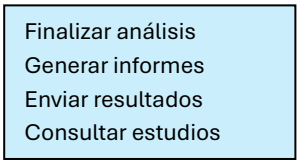
   → generar análisis Clínicos→ compuestos

3. Identificación de muestras
   - El sistema genera un código QR vinculado a los estudios y al paciente.
   - Se imprime una etiqueta con el QR y se adhiere al tubo de muestra.

   → Generar/imprimir QR

4. Extracción de Sangre
   - El bioquímico escanea el QR, verifica los estudios y realiza la extracción.
5. Análisis de Laboratorio
   - El bioquímico vuelve a escanear el QR en el laboratorio.
   - El sistema indica los estudios solicitados por el médico y los reactivos necesarios para llevar a cabo dicho análisis.
   - Se cargan los valores de cada análisis, y cuando se carga el ultimo valor se finaliza el estudio.
   - Algunos análisis, se cargan en forma automática, es decir que tiene una solución que lee el valor del reactivo y lo graba. Los otros reactivos que los procesos de lectura NO están automatizados y el bioquímico, debe colocar el valor en forma manual al sistema.
6. Entrega de Resultados
   - El sistema genera un informe en PDF.
   - El informe se envía por correo electrónico al paciente, también el paciente puede ingresar al sistema web del laboratorio para ver todos los estudios históricos realizados, el paciente ingresa con usuario y clave dado por el laboratorio





Ejemplo de análisis del laboratorio

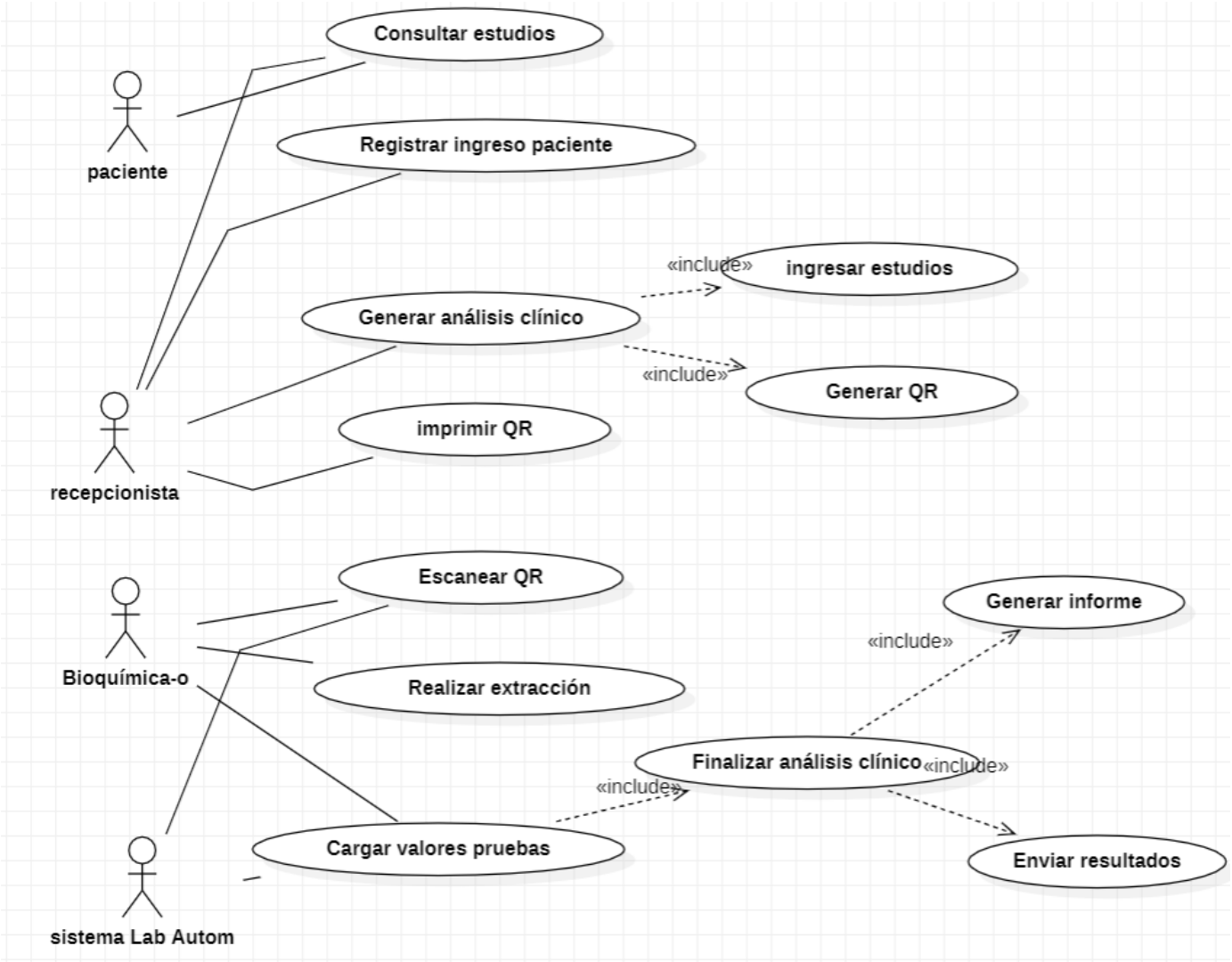
Prueba	Proceso	Reactivo utilizado	Tpo de espera Minutos	Acción luego del tiempo
Colesterol total	automatizado	Reactivo enzimático CT	10	Sistema toma el valor
HDL (colesterol bueno)	automatizado	Reactivo HDL selectivo	12	Sistema toma el valor
LDL (calculado o directo)	automatizado	Reactivo LDL directo	15	Sistema toma el valor
Triglicéridos	MANUAL	Reactivo GPO-POD	35	-

Para comenzar , debemos tener en cuenta lo siguiente:

Un caso de uso tiene un objetivo observable del actor, con valor de negocio

Se recomienda que:

1. *el nombre del CU sea un verbo + objeto, ejemplo Inscribirse en materia, Consultar notas, Generar acta.*
2. *Se debe evitar "hacer clic", "pantalla de…", "grabar", "CRUD de…". Eso es UI/diseño, no objetivo.*
3. *Relaciones:*
   a. *Los **include** (<<include>>) suelen representar sub-tarea obligatoria*
   b. ***extend** (<<extend>>): comportamiento opcional/condicional, colocar la condición entre [Cond]*
4. *Los actores principales en el lado izq, los actores secundarios en el derecho*



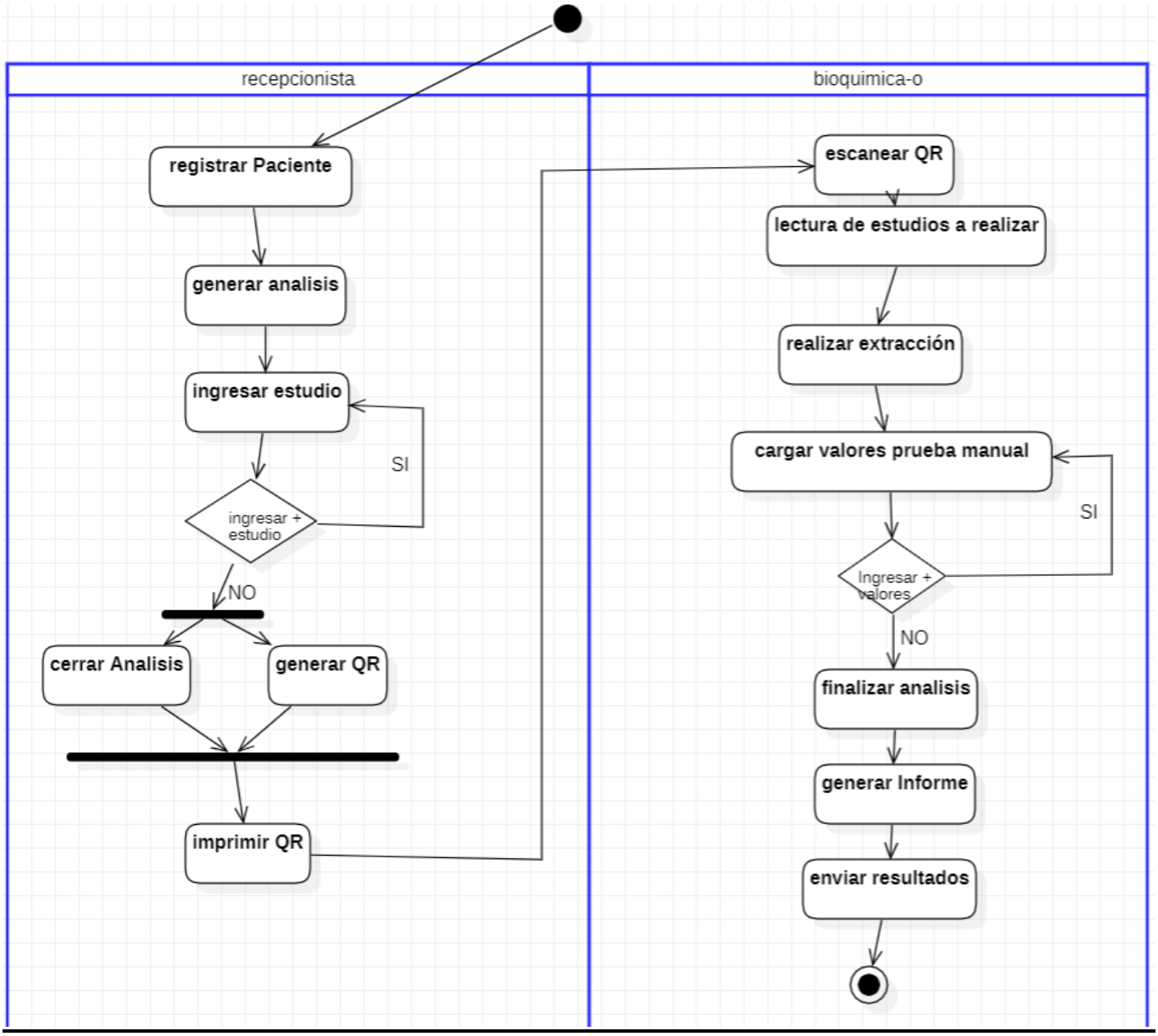
*Otro diagrama que se utiliza para modelar la vista funcional, puede ser el **Diagrama de Actividad,** que muestra el **flujo de trabajo** o **proceso funcional** de una regla de negocio, un caso de uso o una función.*

Tenemos que tener en cuenta lo siguiente:

1. *Todo diagrama debe empezar con un nodo inicial (círculo negro) y terminar en un nodo final (círculo negro con contorno).*
2. *Cada paso es una actividad, de acuerdo al nivel de abstracción que se este manejando , puede ser un proceso, una actividad o una acción.*
3. *Usen verbos en infinitivo: validar, mostrar, enviar.*
4. *Ordénalos según la secuencia, el diag se lee de arriba a abajo.*
5. *Detecten si tienen decisiones, si tienen se representan con nodo de decisión (rombo),*
6. *Incorporen paralelismo si aplica, se representan con Barras horizontales (fork/join) para procesos en paralelo.*

Nosotros vamos a comenzar con un diag de actividad de flujo general del negocio, por lo cual es un nivel de abtraccion alto, entonces seguramente tenemos procesos de negocios y se suele intervenir

varios roles (comprador, delivery, VISA, AFIP) esos roles deben estar en el diag de CU, en este caso es recomendable usar swimlanes para mostrar quién hace qué, es decir ya estamos incorporando RESPONSABILIDAD, Cada lane representa un "responsable".



Otra vista a modelar es la vista de estructura, el diagrama utilizado para el paradigma OO es el diagrama de Clases, de más está en comenzar a definir las clases de dominio (tema visto en análisis y principios de diseño), es recomendable que repasen temas como OO, Clase, UML .

Larman recomienda comenzar a detectar clases candidatas a través de frases nominales en los CU, especificación, requerimientos etc, vamos a volver a leer los CU, y definir las frases nominales(leer Larman).

Nosotros no tenemos especificados los CUs o historias de usuarios (usaría Historias de Usuario, si definí usar un proceso como SCRUM). Solamente tenemos especificado el caso de estudio, por lo tanto vamos a usar ese artefacto de entrada y en el mismo vamos a definir en amarillo las posibles clases ,en verde

posibles funcionalidad / comportamiento, y en celeste posibles atributos o posibles clases (con menos importancia que las clases conceptuales)

PRÁCTICA 1 – UML

Caso de Negocio: Sistema de Gestión para Laboratorio de Análisis Clínicos
El laboratorio clínico recibe diariamente a pacientes con órdenes médicas para la realización de estudios. Actualmente, gran parte del proceso es manual y se desea informatizar para mejorar la trazabilidad de las muestras, reducir errores y agilizar la entrega de resultados.
Objetivo: Automatizar el circuito completo desde el ingreso del paciente con una orden médica, hasta la entrega del resultado.

7. Ingreso del Paciente
    - El paciente se presenta con una orden médica.
    - La recepcionista registra los datos personales y los estudios requeridos.
    - La recepcionista le informa la fecha para retirar el resultado y le imprime la información que es el usuario y clave asignada para ingresar al sitio web
8. Generacion de análisis clínico
    - En el paso previo, la recepcionista registra los estudios requeridos por parte del medico, cada pedido/analisis puede incluir uno o varios estudios específicos (ej. hemograma, glucosa, colesterol, etc.) según lo que solicita el médico.
    - El análisis clínico es diferente para cada paciente, es decir varía según la combinación de estudios solicitados.
      NOTA: Se pide implementar una solución flexible para construir análisis clínicos compuestos a partir de estudios individuales, utilizando un patrón de diseño apropiado para facilitar la extensión y mantenimiento del sistema.
      Explicar que categoría y patron GOF usan y por qué
9. Identificación de muestras
    - El sistema genera un código QR vinculado a los estudios y al paciente.
    - Se imprime una etiqueta con el QR y se adhiere al tubo de muestra.
10. Extracción de Sangre
    - El bioquímico escanea el QR, verifica los estudios y realiza la extracción.
11. Análisis de Laboratorio
    - El bioquímico vuelve a escanear el QR en el laboratorio.
    - El sistema indica los estudios solicitados por el médico y los reactivos necesarios para llevar a cabo dicho análisis.
    - Se cargan los valores de cada análisis, y cuando se carga el ultimo valor se finaliza el estudio.
    - Algunos análisis, se cargan en forma automática, es decir que tiene una solución que lee el valor del reactivo y lo graba. Los otros reactivos que los procesos de lectura NO están automatizados y el bioquímico, debe colocar el valor en forma manual al sistema.
12. Entrega de Resultados
    - El sistema genera un informe en PDF.
    - El informe se envía por correo electrónico al paciente, también el paciente puede ingresar al sistema web del laboratorio para ver todos los estudios históricos realizados, el paciente ingresa con usuario y clave dado por el laboratorio

Ejemplo de análisis del laboratorio

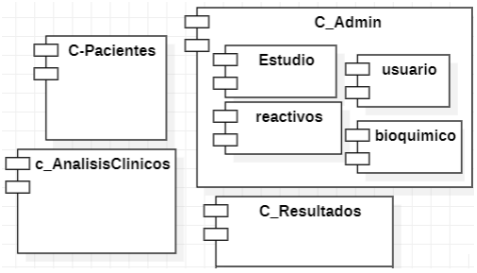
Prueba	Proceso	Reactivo utilizado	Tpo de espera Minutos	Acción luego del tiempo
Colesterol total	automatizado	Reactivo enzimático CT	10	Sistema toma el valor
HDL (colesterol bueno)	automatizado	Reactivo HDL selectivo	12	Sistema toma el valor
LDL (calculado o directo)	automatizado	Reactivo LDL directo	15	Sistema toma el valor
Triglicéridos	MANUAL	Reactivo GPO-POD	35	-

Puede haber más? Es decir hay algunas clases que no contemplamos? … Seguramente, las iremos detectando a medida que vamos iterando.

Clases detectadas: usuario, paciente, pruebas, reactivo, análisis, muestra, SistEntregaResultado, sistExternoOS, etc

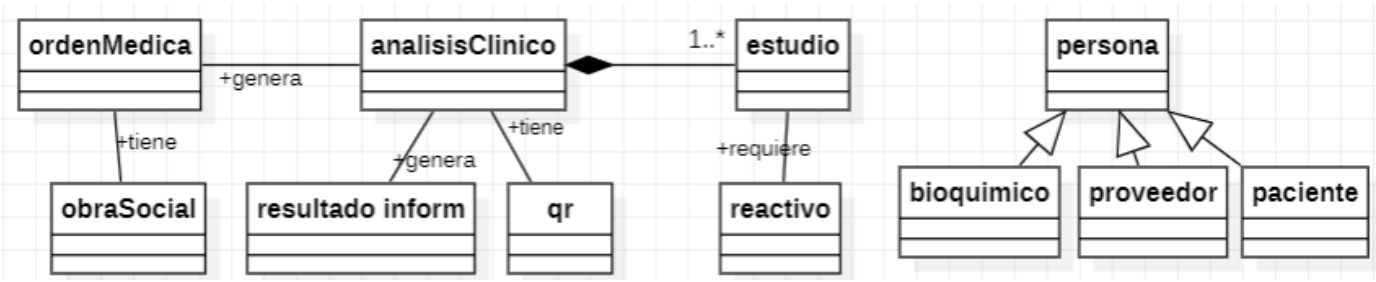
Cuando el arquitecto/analista tiene mucha experiencia ya mentalmente puede pensar cómo puede agrupar los componentes de acuerdo a su naturaleza, esto nos ayuda mucho para incluir en el diseño los principios de diseño como modularidad, separación de incumbencias, si no tienen experiencias para hacerlo mentalmente entonces es recomendable que hagan el diag. de componentes.

El diag. de Comp, inicial es el siguiente:



Vamos a tomar este diagrama como base, luego al finalizar el apunte pueden modificar el modelo, recuerden que el diseño es ITERATIVO y puedo en cada iteración ir agregando mayor nivel de detalle, es decir bajando la abstracción (un nivel de abstracción alto, es una mirada más conceptual y global – ANALISIS, en cambio, un nivel bajo de abstracción nos brinda una mirada detallada y técnica. DISEÑO y nosotros estamos en DISEÑO, y tenemos que llegar a un nivel bajo de abstracción).

Muchas veces generan este tipo de modelo:



Para encontrar mas clases les recomiendo que lean la lista de categorías de clases conceptuales de Larman, (tabla 10.1)

Para colocar el nombre a las relaciones, vean la lista de asociaciones comunes, tabla 11.1

Este diagrama corresponde a un modelo de dominio, con un nivel alta de abstracción, solamente detallo clases de dominio, Si este modelo se lo paso a un programador no le brinda la información de cómo transformarlo a código. Debo aplicar todas las buenas prácticas de diseño:

Detallo algunos de los principios (deben ser estudiados de los libros)

1. Baja Abstracción

- *Significa trabajar cerca de los **detalles concretos** en lugar de conceptos generales, tengo que acercarme al código, no puedo modelar solamente el dominio.*
- *Si no uso este principio puedo tener un sistemas menos flexibles y más rígidos frente al cambio.*

2. Diseño – simple

- *Un **buen diseño** es aquel que resuelve el problema de la manera **más clara y sencilla posible**.*
- *Ejemplo: no agregar 5 capas de herencia cuando un par de clases bien relacionadas alcanza.*

3. Funcionalidad completa

- *El diseño debe **cubrir todos los requerimientos** del sistema, para eso deben evaluar los verbos de la especificación, nosotros lo marcamos en color verde.*
- *No sirve un diseño "lindo" si deja cosas sin resolver.*

4. Usar el estándar de UML

- *UML es el **lenguaje visual estándar** para modelar sistemas OO.*
- *Beneficio: **todos entienden lo mismo** (arquitectos, devs, testers). Si cada participante realiza una versión propia de UML, entonces no se cumple el objetivo principal de UML que es transmitir la solución a implementar*

5. Diseño orientado a objetos

Entender los conceptos de:

- ***Clases** → definen estructura y comportamiento (atributos + métodos).*
- ***Objetos** → instancias concretas de clases.*
- ***Herencia** → compartir y reutilizar comportamiento.*
- ***Mensajes** → cómo los objetos colaboran (métodos llamados).*

6. Separación de incumbencias

- *Principio: dividir el sistema en **partes más pequeñas** y claras, cada una con su responsabilidad.*
- *Ejemplo: separar la lógica de negocio (ventas) de la interfaz (pantalla) y de la persistencia (base de datos).*
- *Beneficio: menor complejidad, más entendible y **mantenible, y si es mantenible puedo lograr tener agilidad al cambio y ahorro de dinero en el mantenimiento del sistema**.*

7. Modularidad

- *Un sistema debe estar compuesto de **módulos independientes** que se puedan entender y modificar sin romper todo. Asociar los componentes de acuerdo a su naturaleza en común.*
- *Ejemplo: un módulo "inventario" no debería compartir componentes con el módulo "SueldoEmpleado".*

8. Refinamiento

- *Detallar lo mas posible, agregar atributos, métodos, restricciones, relaciones más específicas, multiplicidad, nombre a las relaciones, etc etc*

9. Patrones GRASP

a) ***Experto en la información***

   a. ***Principio**: asignar la responsabilidad a la clase que tiene la información necesaria para cumplirla.*

b) ***Creador***

   a. ***Principio**: una clase debe ser responsable de crear objetos de otra clase si la contiene, o la usa intensivamente, o tiene los datos para inicializarla.*

c) ***Controlador***

   a. ***Principio**: asignar la responsabilidad de manejar un caso de uso del sistema a un objeto controlador que representa, en nuestra caso para el examen vamos a usar un controller por componente/modulo*

   b. *Nota de Larman*

Un error típico del diseño de los controladores es otorgarles demasiada responsabilidad.

> Normalmente, un controlador debería *delegar* en otros objetos el trabajo que se necesita hacer; coordina o controla la actividad. No realiza mucho trabajo por sí mismo.

ser responsables de llevar a cabo los eventos del sistema. En otras palabras, las operaciones del sistema se deberían manejar en la lógica de la aplicación o capas del dominio en lugar de en la capa de interfaz del sistema. Diríjase a la sección "Cuestiones y Soluciones" para ver un ejemplo.

d) ***Bajo acoplamiento***

   a. ***Principio**: diseñar para que las clases dependan lo menos posible unas de otras, me ayuda en el mantenimiento, la reutilización y la agilidad al cambio.*

e) ***Alta cohesión***

   a. ***Principio**: cada clase debe tener responsabilidades claras, relacionadas y limitadas. Eviten las "clases TODA-PODEROSAS " que hacen de todo, muchas veces hacen gestores que tiene toda la funcionalidad de la solucion.*

f) ***Polimorfismo***

   a. ***Principio**: manejar comportamientos variantes a través de herencia o interfaces.*

g) ***Fabricación pura***

a. ***Principio****:crear una clase artificial (no del dominio) para cumplir la responsabilidad. PRINCIPIO MUY UTILIZADO EN EL DISEÑO*

h) ***Indirección***

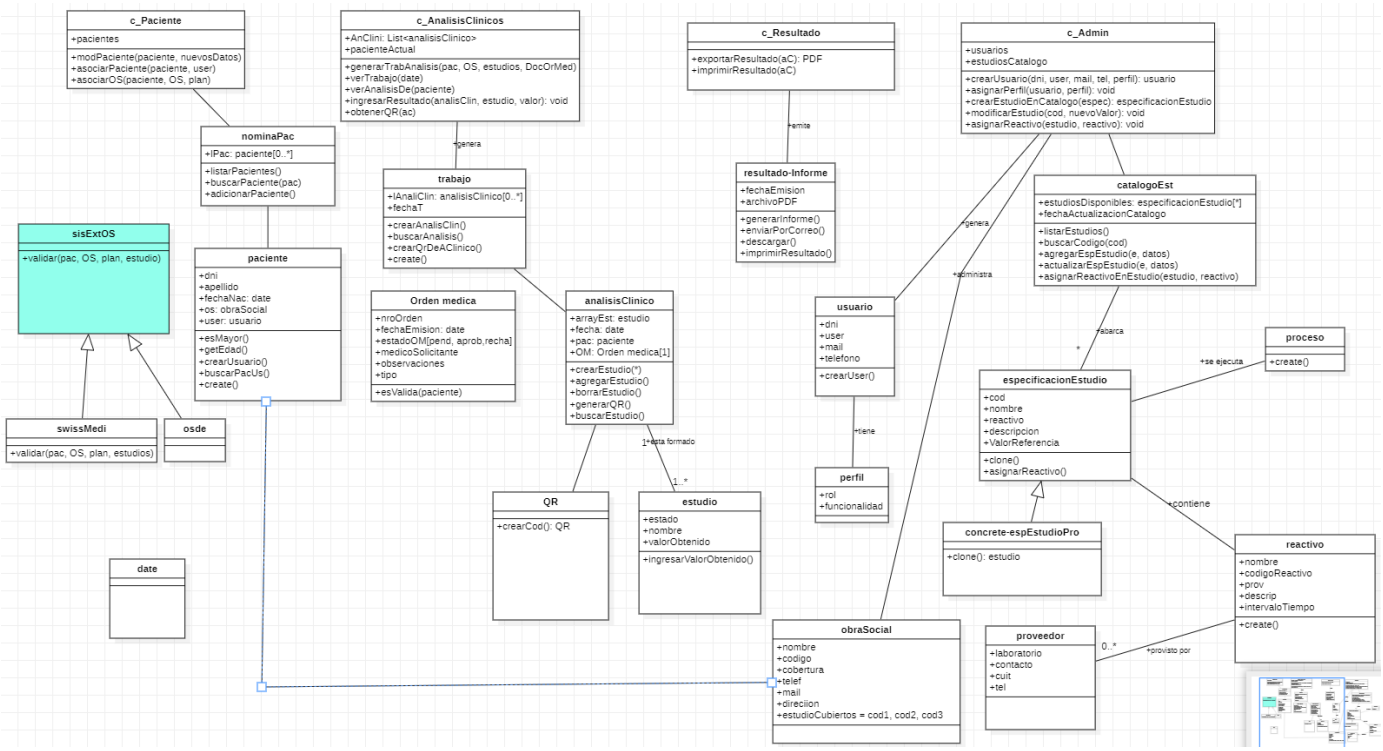
a. ***Principio****: usar un intermediario para reducir acoplamiento entre componentes.*

i) ***Variaciones protegidas***

- ***Principio****: proteger elementos del sistema de los efectos de cambios en otros, creando* ***interfaces estables****.*

10. Patrones GOF

Diagrama de clases



Ubicamos en la parte superior los controller, modelamos la solución como si fueran capas(controller-modelo de negocio y abajo estaría los objetos que acceden a los datos, que no se modelan porque excede el alcance de la materia). Si usan gestores deberían estar en esa capa de los controler, luego seguimos con las clases mas importantes de negocio. Es MUY importante la ubicación de las clases, porque la lectura en este diagrama es de arriba hacia abajo y de izq a derecha. Las clases con la misma naturaleza las ubico cerca para una mejor lectura, (ayuda mucho definir antes el diag de componente, para llevar a cabo ese ordenamiento).

Este diagrama debe tener: nombre de las asociaciones, multiplicidad, navegabilidad (en este diag no lo puse porque respeto la lectura de UML), atributos, comportamiento definido en el caso de estudio, se respeta la escritura de UML

NOTA: Hay una clase en verde porque esa clase se relaciona con un sistema externo, es mi puerta de salida para comunicarme con sistemas externos

Diagrama de Secuencia / comunicación.

Debemos pensar en un escenario para modelar, si no tengo agilidad para pensar en escenarios me puede ayudar pensar cómo sería la pantalla que lanza el evento que captura el controller, les dejo la interfaz donde la recepcionista carga información sobre el paciente

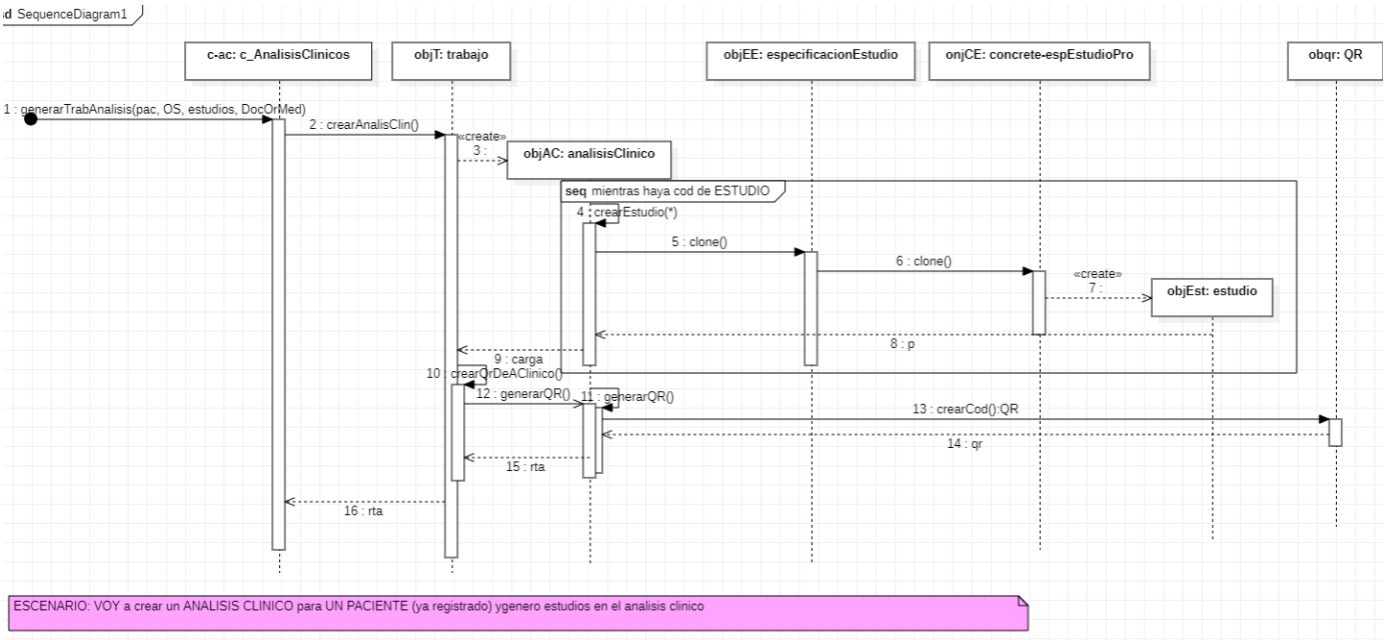


Debo pensar en cómo llevan a cabo dicha funcionalidad. Asumo que la recepcionista lee estudio (pedido del medico) y coloca toda la información en una interfaz (como la que esta modelada) y hago clic en el botón ingresar, ese evento, lo captura el controller acorde al CU o componente, en este caso el controller C-AnalisisClinicos, podría pasarlo a un gestor, pero tendría los mismos métodos que el controller.
Yo saque el gestor, para simplificarlo y el controller tiene la lógica de direccionar el pedido al modelo. Es de buenas practicas de diseño que el controller solo capture el evento y tenga una lógica MINIMA, su responsabilidad es capturar el evento y pasar la responsabilidad al MODELO de negocio.
Defino el escenario y comienzo a modelar el Diag de secuencia.
Saco el actor, el mensaje es de un detinatario anonimo , porque puede ser que el evento lo hizo la recepcionista o recibí un pedido de un servicio, en esta vista no me interesa el actor, eso ya se entendio en el análisis, acá debo entender que el emisor puede venir de diferentes fuentes , por esp coloco un mensaje anonimo ,generarTrabAnalisis(parametros), que lo captura el controller de c_analisiClinicos, el controller debe tener lógica mínima del negocio, solo es un director de orquesta. Si agrego un gestor, el controller solo captura, si no tengo gestor , le adiciono una lógica mínima, yo tome la decisión de sacar el gestor, entonces asumo que el controller va a tener una lógica mínima. Pienso y defino el siguiente escenario:

- *Envio evento , y lo captura el controller*
- *Si ya tengo un **trabajo** para el día de hoy, lo uso ,, y genero un **análisis clínico** (analisisClinico).*
- *Ese análisis necesita crear varios **estudios**.*
- *En lugar de construirlos "a mano", voy a decir que el sistema **clona las especificaciones de estudio** (especificacionEstudio).*
- *Cada especificación concreta (concrete-espEstudioPro) implementa clone(), devolviendo un **nuevo estudio** con los atributos básicos ya cargados (nombre, tiempo, reactivo).*

  *Este escenario me da la idea de usar el **patrón Prototype (GoF)**, lo cual me permite crear nuevos objetos **copiando** (clonando) otros objetos existentes.*
  Se usa cuando los objetos a crear comparten gran parte de su estructura y solo cambian en detalles.

*NOTA: clone()devuelve **un nuevo objeto independiente** y ese nuevo objeto **no está atado al original, se puede c**ambiar los valores .*



Los mensajes que modele en el diag. de interacción, deben estar contemplados en el diag de clases, ambas vistas deben estar integradas. Si hay mensajes en el diag. de interacción que no están en el diag de CLASES, el ejercicio esta MAL

Por eso agrego este cambio en el diag. de clase

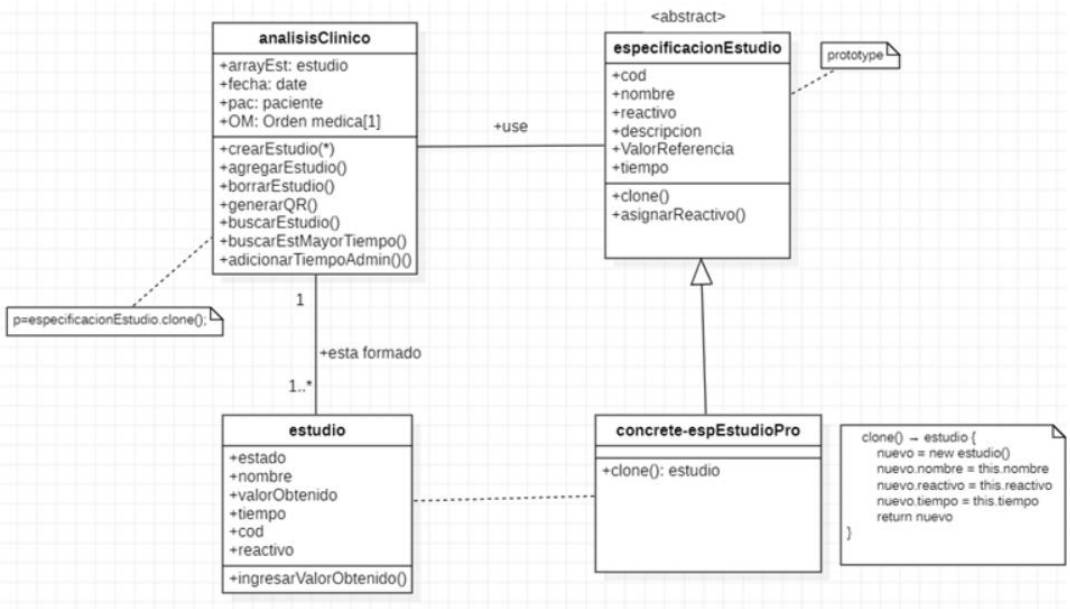


Diagrama de Estado

Un diagrama de estados muestra el cambio de estado de un objeto. Se compone de estados, que representan las situaciones en las que puede encontrarse el sistema; transiciones, las flechas que indican el paso de un estado a otro; y eventos, los desencadenantes que inician esas transiciones.

Los elementos del diag de estado son:

• Estados: Se representan con rectángulos de esquinas redondeadas

• Transiciones: Son las flechas que conectan los estados, indicando el camino de un estado a otro.

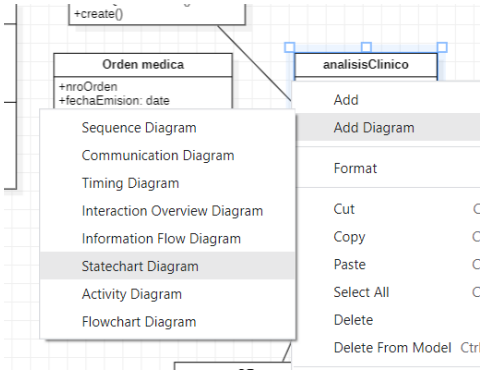
• Eventos: Son los estímulos, internos o externos, que activan una transición de un estado a otro. Se suelen etiquetar sobre la flecha de transición.

• Estado inicial: círculo oscuro que indica el punto de partida de la máquina de estados.

• Estado final: círculo con un punto en el interior que señala que el proceso ha concluido

Para que valga la pena modelar el diag de estado hay que pensar en un objeto que realmente sufra cambios de estados en su vida, vamos hacer el Diagrama de Estado del objeto analisisClinico,

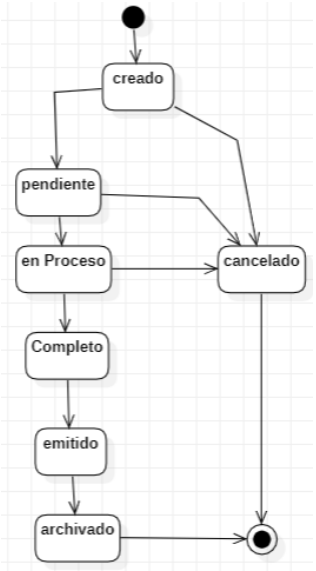
Aclaración, si usan el staruml, se posicionan en la clase del diag de clase, botón derecho y agregar Diagrama de estado, para que esté relacionado con dicho clase



Les recomiendo que piensen qué estados posibles puede transitar ese objeto:

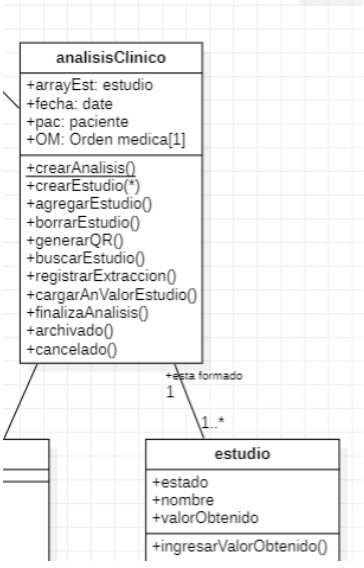
Estados posibles:

• **Creado** (cuando se genera desde la Orden Médica)
• **Pendiente** de Extracción (esperando ser llamado para tomar la muestra)
• **En Proceso** (extracción hecha, y tomando y cargando valores de los estudios)
• **Completado** (todos los estudios ya tienen valores)
• **Emitido** (PDF disponible y enviado)
• **Archivado** (almacenado en histórico del paciente)
• **Cancelado** (cancelado por motivos administrativos o técnicos)



Para estas Transiciones, debemos pensar qué comportamientos se deben ejecutar para hacer este cambio de estado:
Podemos pensar en los siguientes métodos:
• **Creado** → crearAnalisis()
• **Pendiente** registrarExtraccion()
• **En Proceso** cargarAnValorEstudio()// cuando se carga el primer valor
• **Completado** cargarAnValorEstudio()// cuando se carga el ultimo valor
• **Emitido** finalizarAnalisis()
• **Archivado** archivar// proceso batch
• **Cancelado** cancelado (motivo)// cancelado por diferente motivo: admin, problemas con OS(por ejemplo la OS manda una cancelación), por temas técnicos (por ejemplo no hay reactivo)

Estos comportamientos deben estar reflejado en el Diag de clase
No perdamos de vista que el diagrama de estado se modela para el objeto **analisisClinico** y es el propio objeto que debe ser responsable de transicionar entre estados, porque él es el experto de conocer la información de cambiar el estado (analisisClinico conoce por ejemplo: cantidad de estudios cargados (es contenedor de estudios a realizar), estado actual, condiciones (por ejemplo condiciones si quieren cancelar el analisis))
Existen otros objetos que pueden invocar métodos, como por ejemplo el objeto TRABAJO, que tiene el método CrearAnalisisClin, invoca, pero no cambian atributos internos directamente del objeto de **analisisClinico**.



ACLARACION:
El objeto analisisClinico debería ser inicializado con estado = "Creado".
Eso lo puede manejar directamente el constructor (por defecto o parametrizado).
Trabajo puede invocar AnalisisClinico.crearAnalisis(...) o usar new AnalisisClinico(...).
Pero el cambio de estado a "Creado" siempre lo hace el propio AnalisisClinico, no Trabajo.
Así mantenemos el patrón Experto: cada objeto controla su propio estado.

Diag comunicación:

*Recordar que deben escribir **siempre** el escenario que van a modelar*

Escenario: Cancelar análisis por falta de reactivo,
Es preferible DETALLARLO por demás,, ejemplo:
El bioquímico / recepcionista/ dueño del laboratorio/ o la maquina que automatiza el proceso,, no importa quien lo ejecuta.. lo importante es que tenemos un evento que es capturado por el controller c_AnalisisClinico
Luego analisisClinico debe realizar:
* cancelarAnalisis(numAnalisis, motivo)// Colocar el motivo a análisis
* cambia estado = Cancelado
* Eliminar los estudio que pertenecen al análisis// destroy().// *este paso lo puse para tener un método más.*

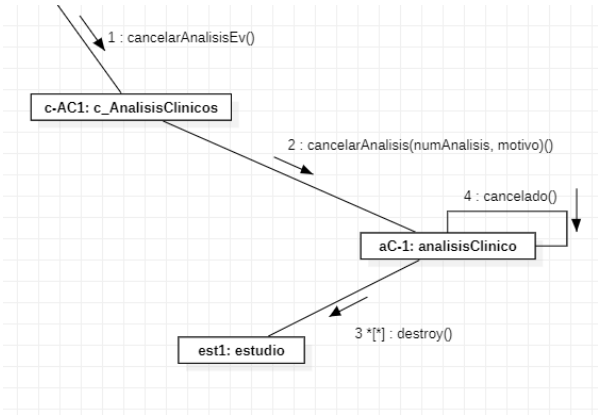
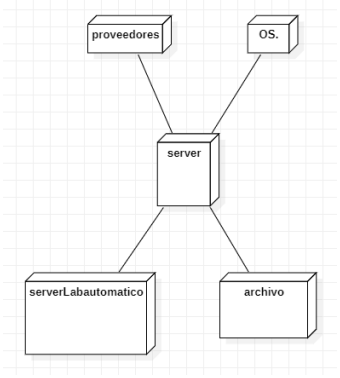
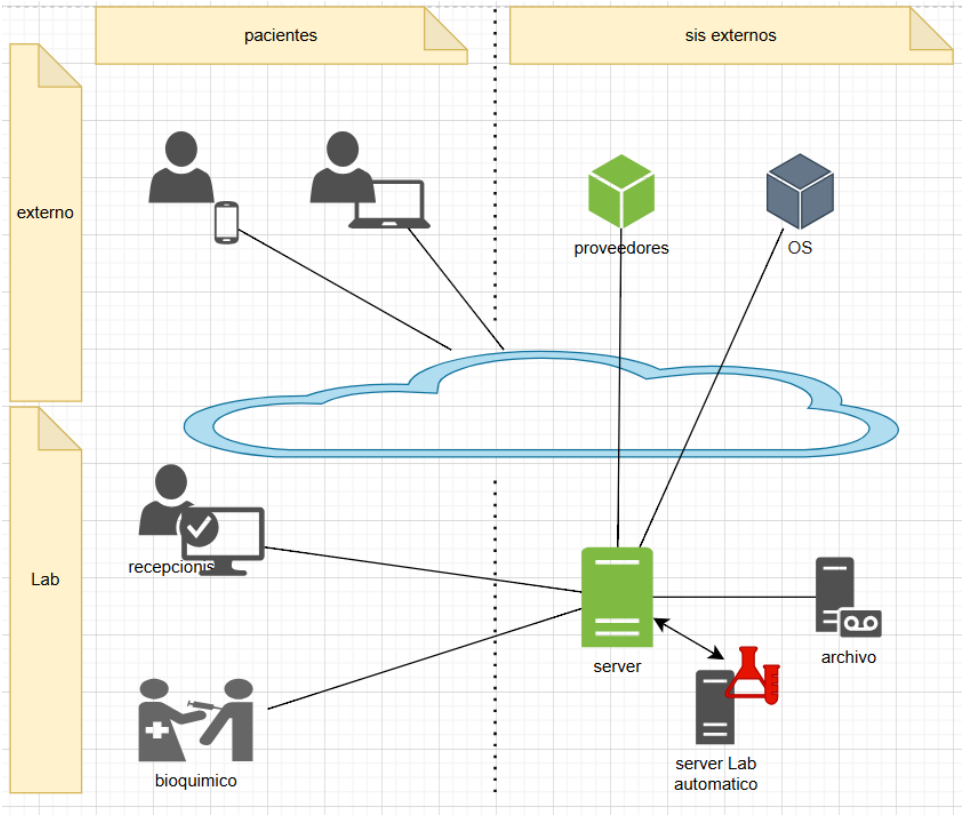


Diagrama de Despliegue

El diagrma de despliegue muestra una vista de la infraestructura de la solucion propuesta, para modelarlo hay que tener mucho conocimiento de redes de hardware , que es un conocimiento que excede nuestra materia, pero debemos modelar un diagrma aunque no sea tan detallado
Vamos a modelar dos modelos
El primero con uml 2.0, en el cual los elemnetos son los nodos y la comunicación, Nosotros debemos colocar los nombres que representan esos nodos.

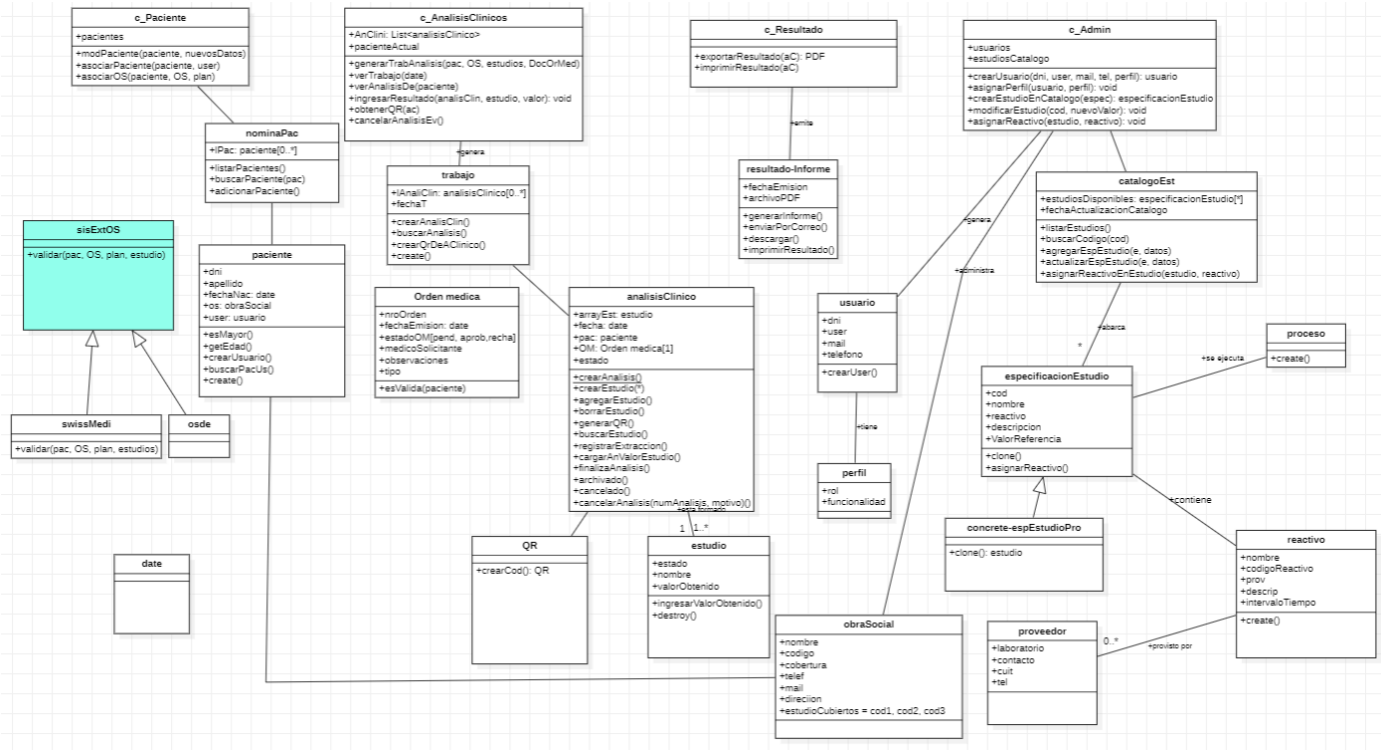


El segundo diagrama nos basamos en una extensión de elementos que no están en UML, pero representan elementos físicos de una posible infraestructura



Ambos diagramas de despliegue son del mismo sistema, pero con distintos niveles de detalle.

El primero solo se centra en la infraestructura técnica (servidores, conexiones), no distingue a qué dominio pertenece, son solo nodos conectados. Y el segundo diagrama, muestra actores humanos (pacientes, bioquímico, recepcionista), sistemas externos (OS, proveedores), componentes internos (server, serverLab automático, archivo) , modelamos la separación de los dominios y responsabilidades, es decir el segundo ayuda a entender quién usa qué nodo y cuál es la responsabilidad de la organización(laboratorio) y cual es la responsabilidad de serv. Externos. El segundo es más fácil de entender.



Los modelos son representaciones que permiten comprender y organizar la solución antes de implementarla. Debemos recordar que son iterativos y evolutivos, en cada iteración se pueden enriquecer con más detalles y refinamientos. Por eso, modelar no es un paso único, sino un ciclo continuo de mejora.

CONSULTAS DE ALUMNOS

Durante estos años de dar diseño, observe que los alumnos siempre tienen dudas similares. Por eso considero útil colocar las preguntas de los alumnos y responderlas, (las consultas están en color negro y están tal cual las hicieron los alumnos, las respuestas están en color azul):

DUDA: ¿No sé cómo poner las asociaciones en el modelo? Yo siempre pondría una clase con muchas asociaciones porque para mí, las clases están todas conectadas

Para poder manejar las asociaciones en el diag de Clase y tener bajo acoplamiento, sí o sí tenemos que relacionarlo con otro concepto que es la “visibilidad”

La Visibilidad es la capacidad que tiene un objeto para "ver" o acceder a otro objeto. En otras palabras, si el objeto A puede enviarle un mensaje al objeto B, eso significa que B es visible para A. (LARMAN CAPITULO 18)
Este concepto es clave cuando diseñamos la comunicación entre objetos (en diagramas de secuencia o de comunicación), porque un objeto no puede comunicarse con otro si no lo puede "ver".
Entonces ¿Cómo puede un objeto A tener visibilidad del objeto B?
Siempre volvemos a Larman,, en el libro menciona cuatro formas comunes de lograr esta visibilidad: // *copio tal cual el libro*

1. **Visibilidad de atributo:**
   El objeto B es un atributo (campo) del objeto A.
   Ejemplo: this.catalogo si catalogo es un atributo del objeto Registro.
2. **Visibilidad de parámetro:**
   El objeto B es un parámetro de un método del objeto A.
   Ejemplo: void procesar(CatalogoDeProductos catalogo) { ... }
   A puede usar a B dentro del método.
3. **Visibilidad local:**
   El objeto B es una variable local (no parámetro) dentro de un método de A.
   Ejemplo: CatalogoDeProductos catalogo = new CatalogoDeProductos();
4. **Visibilidad global:**
   El objeto B es visible de forma global, es decir, accesible por muchos objetos sin necesidad de ser pasado como parámetro o atributo.
   Esto puede implicar usar patrones como Singleton, aunque se recomienda con cuidado para evitar acoplamientos fuertes.
   Para que un objeto A envíe un mensaje a un objeto B, B debe ser visible a A.
   Sin visibilidad, no puede haber comunicación.
   Cuando modelan las interacciones entre objetos, como en los diagramas de secuencia o comunicacion, sólo pueden conectar objetos que tienen visibilidad entre sí. Esto también influye en cómo modelan las relaciones entre clases (atributos, parámetros, etc.).
   En el Diagrama de CLASE solo vamos a ver (en una primera instancia), la Visibilidad de atributo, si detallan los métodos podemos ver la segunda,, pero después vuelvo a esto.
   Cuando hay una asociación entre clases en un diagrama de clases UML (una línea entre clases), por defecto representa visibilidad de atributo.



Esa línea indica que venta tiene un atributo (o al menos una referencia) a una instancia de línea de Venta.
Entonces, en UML, esta asociación se traduce como visibilidad de atributo.
Las otras visibilidades NO se reflejan directamente en el diagrama de clases, excepto que estén MUUUY DETALLADAS, que significa muy detalladas? cuando hay visibilidad de parámetro.

Paso a dar mas detalles de las visibilidades para entender

- **Visibilidad de parámetro:** solo aparece en la declaracion del método (código o especificación detallada).si en el diagrama de clases ustedes colocan los métodos y definen los parámetros y su tipo,, pueden ver esta visibilidad,, pero no es taaan intuitivo como la visibilidad de atributo
- **Visibilidad local:** es una variable declarada dentro del método, y eso es solo visible en código o pseudocódigo, no en diagramas UML., si usa esta visibilidad pueden definirlo en una nota en el diagrama de clase, sec, o comunicación
- **Visibilidad global:** rara vez se ve en un diagrama de clases. Si usán un patrón como Singleton, podría deducirse, pero no está explícitamente modelado como "visibilidad" en UML.

Resumen

- Diagrama de clases → solo muestra visibilidad de atributo (asociaciones). En el mejor de los casos si hay detalle podemos ver visibilidad de parámetro
- Diagrama de secuencia → muestra qué objeto envía mensaje a cuál, por tanto, ayuda a deducir si se pasa como parámetro, se crea localmente o un atributo.
- Código → es donde se ve todo explícitamente.

RESUMIENDO

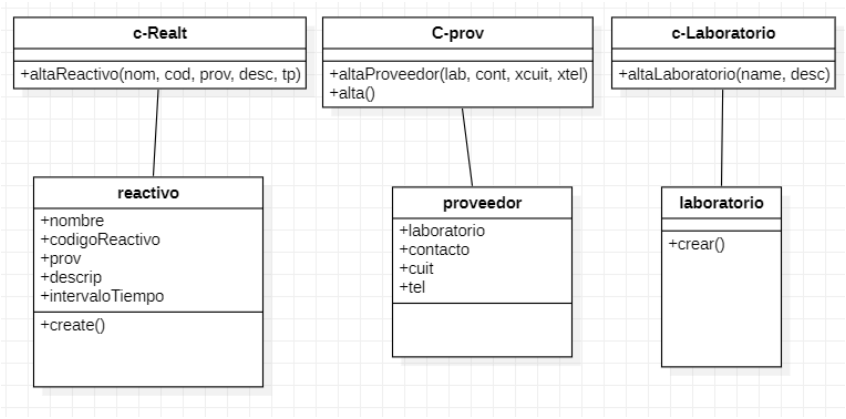
Tipo de visibilidad	¿Se ve en diagrama de clases?	¿Se ve en diagrama de secuencia y comunicacion?	¿Se ve en código?
Atributo	Sí	Sí (si se usa)	Sí
Parámetro	depende de como lo modelen	Sí (si se pasa por mensaje)	Sí
Local	No	Sí (si se crea dentro)	Sí
Global	No (salvo diseño especial)	Sí (si se accede globalmente)	Sí

Ahora sí,,,, respondo lo que siempre preguntan,,,, **¿Qué asociaciones poner en el diagrama de clases?**

- Deben poner las asociaciones que realmente necesitan para que los objetos colaboren entre sí en los diagramas de secuencia/comunicación.
  Es decir, si en un diagrama de secuencia, el objeto A le manda un mensaje a B, entonces A debe tener visibilidad de B, y si esa visibilidad es por atributo, hay que reflejarla como una asociación en el diagrama de clases.
- Deben también colocar las Asociaciones significativas del dominio o negocio, aunque no aparezcan (todavía) en un diagrama de secuencia / comunicación.
- No pongan asociaciones innecesarias "por si acaso", porque hacen que el modelo se vuelva confuso y difícil de mantener, piensen en su diag de clase como una foto para que otra persona pueda entender lo que ustedes quieren transmitir, si todo es IMPORTANTE en el diag. de clase y ponen todas las asociaciones por las dudas, entonces no es UTIL, no me transmite NADA…

DUDA : Realice el modelo y me quedo un controller por cada CLASE que modele, Y no lo veo en sus ejemplos, entonces está bien o está mal?

Es decir realizan esto(ver imagen).



Una clase <–> un controler, colocan un controller por cada clase, ejemplo para la clase reactivo lo relacionan con un controller, el motivo que explican el porque lo modelaron de esta forma ,, es que hay funcionalidad donde un administrativo(actor) da de alta el Reactivo, ese evento lo captura Controller del Reactivo

La realidad es que muchas clases en un sistema van a tener operaciones de **Alta/Baja/Modificación (ABM).** Sin embargo, no siempre vale la pena modelarlas en detalle en los diagramas.
¿Por qué? Porque si me pongo a modelar todos los ABM, básicamente estaría diseñando lo mismo que luego se programa casi de manera “automática”. Eso no aporta mucho valor en la etapa de diseño de arquitectura. En esta etapa lo que buscamos es modelar **el core del problema, lo más crítico, lo más complejo, lo que realmente necesita ser definido para que el programador lo entienda y lo implemente bien.**
El ABM, en cambio, es algo que un programador debería poder resolver sin necesidad de un diseño detallado previo.
Ahora bien, no está mal si quieren incluirlos, especialmente en equipos donde alguien se dedica solo a programar ABMs. En ese caso puede ayudar, pero al mismo tiempo complica y hace más pesado el diseño que debería enfocarse en lo esencial (el modelo que nosotros hacemos, debemos plantear la solución)

- Los Alta/Baja/Modificación (CRUD) son operaciones **repetitivas y mecánicas**, cuya lógica suele estar **estandarizadas** y en muchos frameworks incluso son **automáticas** (ej: en Django, Spring, .NET, etc. ya hay generadores de CRUD).
- Por eso, en general **no aportan complejidad ni decisiones de diseño**: son simplemente persistencia y validaciones básicas.
- El valor del diseño está en **resolver lo crítico del dominio**:
    - Cómo se relacionan los objetos.
    - Cómo se distribuyen las responsabilidades (GRASP).
    - Qué controladores definen los flujos principales.
    - Qué patrones aplicamos para mantener bajo acoplamiento y alta cohesión. etc etc

- Ejemplo: En un sistema de estacionamiento medido, lo central no es “dar de alta un usuario”, sino **cómo se valida el pago, cómo se controla la ocupación de un espacio, cómo se gestionan**

las multas. En esto nos enfocamos en la materia, que el alumno pueda entender lo que es realmente importante a modelar.

Cuando tiene sentido modelar ABMs:

- Si el equipo es muy heterogéneo y tienén programadores que solo saben implementar ABMs, un diseño UML de esas operaciones **puede servir como guía**.
- Pero en contextos académicos o profesionales con cierta experiencia, **modelar todos los ABMs solo recarga el diagrama y oculta lo realmente importante además que es un gasto innecesario de costos, ya que modelar algo trivial cuesta dinero y tiempo**.

  En resumen:
  - No hace falta modelar los controllers solo para cumplir con la funcionalidad de ABM.
  - **Lo importante es modelar la lógica central del negocio.**

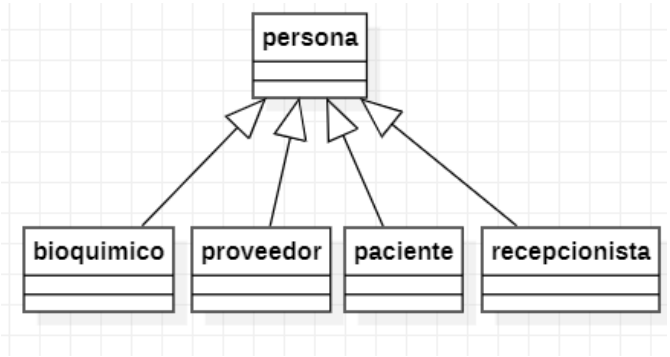
DUDA: ¿En este tipo de ejercicios donde se describe la funcionalidad a través de varios roles, como sé cuáles tienen importancia para modelarse en el sistema o deberían modelarse todos? El rol de recepcionista y el de bioquímico, no entiendo si tienen un valor anecdótico o son de vital importancia en este ejercicio.

Muchas veces se confunden **roles, cuando en realidad son actores (actores que modelamos en los diagramas de Casos de Uso**) y solemos colocar esos actores como **clases** en el **diagrama de Clases**. Esto es comprensible en la etapa de análisis, porque estamos en un nivel alto de abstracción y todavía estamos entendiendo el problema. Ahora bien, cuando bajamos el nivel de abstracción y vamos detallando más el diseño, vemos que esos roles no siempre deben transformarse en clases.

La clave está en preguntarse:

- ¿Este rol corresponde a una **entidad** del negocio?
- O ¿es simplemente un **actor** que interactúa con el sistema?

Por ejemplo, volvamos al ejercicio de laboratorio que estamos trabajando, y pensemos que estamos en una etapa de análisis y realizamos el siguiente modelado:



Pensemos en la clase recepcionista, que en los CU tienen la responsabilidad de interactuar con el sistema y realizan funciones como: dan de alta al paciente, leen la orden médica, generan el análisis clínico, validan con la OS si el paciente es socio, etc etc, entonces esa idea de ver recepcionista como un actor en los CU, nos lleva a pensar en modelar la clase RECEPCIONISTA en el diag de clases, y

colocamos en dicha clase los métodos que lanzan el CU, puede ser altaDePaciente, validarObraSocialHabilitada (métodos generadores de evento), En DISEÑO es un error.

Recepcionista o secretario/a o empleado administrativo, quien sea,,, no me importa quien genera ese evento, es solo un actor que **usa** el sistema, pero no es fundamental para el flujo del negocio. Ese rol se modela en el diagrama de casos de uso, pero no necesariamente como clase en el diagrama de clases, en cambio **Paciente** sí debe modelarse como clase porque es parte del **core del negocio**. Todo análisis o estudio está asociado a un paciente, por lo que debe existir en el modelo.

Una ayuda para evitar este tipo de errores en el diseño es preguntarse lo siguiente

Preguntas para detectar si es una clase y no un actor

1. ¿Necesito **guardar información** de este objeto en el sistema?
   (Ejemplo: nombre, DNI, historial → Paciente)
2. ¿Se relaciona con otras entidades para dar soporte al negocio?
   (Ejemplo: Análisis está asociado a un Paciente)
3. ¿Tiene **comportamientos propios** dentro del dominio?
   (Ejemplo: un Paciente puede pedir turno, tener análisis, etc.)
4. ¿Es parte del **core del negocio**?
   (Si desaparece, el sistema pierde sentido → sin paciente no hay análisis).

Preguntas para detectar si es Actor y no una clase

1. ¿Su función es **usar el sistema** más que formar parte de él?
   (Ejemplo: recepcionista que carga datos en el sistema).
2. ¿Podría ser reemplazado por otra persona/rol sin que cambie la lógica del negocio?
   (Ejemplo: recepcionista ↔ secretario↔ dueño del laboratorio).
3. ¿No necesito guardar datos permanentes de él como **entidad central** del negocio?
   (Sólo importa el usuario que interactua con el Sistema).

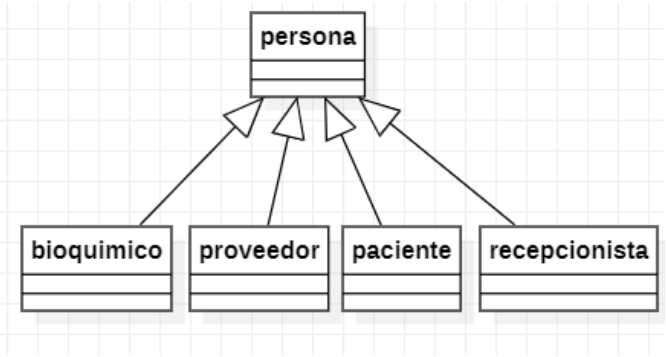
En resumen: **no todos los roles son clases**. Los que representan entidades del dominio del problema (como Paciente) sí se modelan; los que representan actores externos (como Recepcionista) quedan en casos de uso.

- *“Si necesito que esté en la solución con atributos, comportamiento y relaciones → es Clase (Entidad).”*
- *“Si solo entra, usa el sistema y se va → es Actor.”*

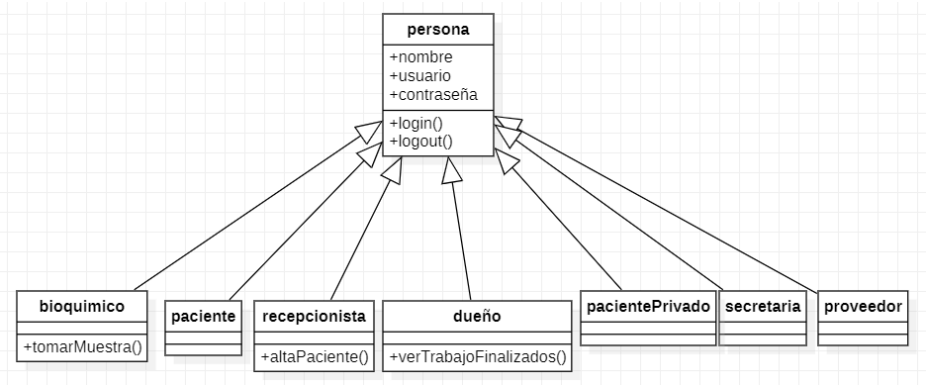
Otro punto muy importante aclarar con esta duda es que si el sistema incluye un **módulo de administración de usuarios** (por ejemplo, gestión de login, perfiles, permisos), ahí **sí pasa a ser parte del modelo, pero no es parte del CORE del negocio, por eso no lo incluimos en los TPs**

Pensemos en un **módulo de administración de usuarios** para el caso del laboratorio, en la etapa de análisis pienso en clases como recepcionista, bioquimico, paciente (para que pueda ver sus resultados), pero en diseño tengo que aplicar el concepto de **Extensibilidad, s**ignifica que el sistema está pensado de tal forma que, cuando el negocio cambie o aparezcan **nuevas funcionalidades**, se pueda **extender** (agregar cosas) sin necesidad de reescribir ni romper lo que ya funciona.

Entonces si hago esto:

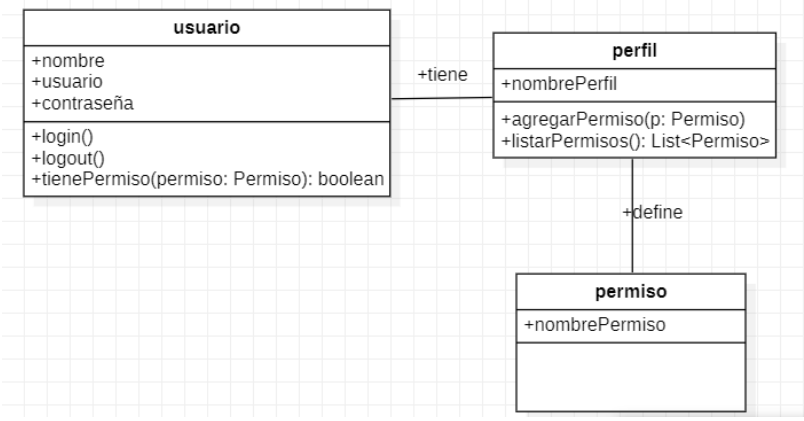


Y luego quiero agregar más actores, podría tener un problema de la estructura porque debo agregar más entidades



Este modelo no respeta los buenos principios de diseño algo más correcto sería pensar en

- **Usuario** → datos básicos (nombre, usuario, contraseña).
- **Perfil** → define el rol (Recepcionista, Bioquímico, etc.).
- **Permiso** → acciones que puede realizar cada perfil (ej. registrar turno, validar resultados).



El diagrama es un ejemplo, y puede ser mejorado

DUDA: En el caso del diagrama de clases, siento que puedo arrancar los ejercicios, pero llega un punto en el que no sé cómo continuarlo al no entender al 100% el dominio, y me doy cuenta de que me quedan clases con pocos atributos básicos (id, descripción, nombre) y que los métodos terminan siendo crear, modificar o eliminar. ¿Como puedo identificar mejor estas partes del diagrama y ver lo más importante?

Es normal que muchas clases en un sistema tengan solo id, nombre, descripción. Pero eso responde a la parte administrativa del sistema, y no siempre vale la pena profundizar en su diseño porque el programador puede implementarlo casi directo con un ABM.
Lo importante en diseño es identificar las clases que hacen que el negocio funcione:

- En un laboratorio → Paciente, Análisis, Resultado.
- En un sistema de ventas → Venta, Producto, Pago.
- En estacionamiento medido → Ticket, Vehículo, Agente.

Si ven que todas las clases terminan en crear(), modificar(), eliminar(),get(), set(), es señal de que <u>están en un nivel muy superficial.</u>
Deben pensar en acciones del negocio: validarResultado(), emitirFactura(), calcularMonto(), generarTicket(), reservarTurno().

Consejos:

- relean la especificación o caso de estudio, detecten *qué clase colabora y que funcionalidad hay ,, seguramente son CORE*, ahí aparecen los métodos fundamentales.
- Usen los patrones GRASP como guía
- Leer la lista de clases más usadas del libro de Larman
- Para el TP de la materia, deben iterar, es normal que al principio las clases se vean vacías, el diseño se enriquece a medida que iteran

DUDA: profe le hago una consulta, leyendo el libro de Larman encontré que efectivamente para que una clase A quede asociada a través de atributo con otra clase B la primera tiene que recibir una referencia de la segunda, mi pregunta es cómo se obtiene esta referencia? porque nosotros generalmente tomamos atributos específicos de una clase (el ID por ejemplo), no la referencia de una instancia.

Antes voy aclarar lo siguiente , tengo el siguiente cod**:**

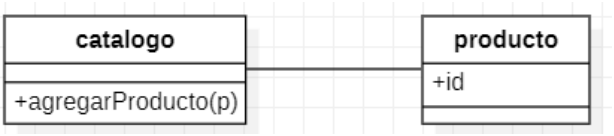
class Venta {
int idProducto;
}
// Acá paso solo un valor
	class Venta {
	Producto producto;
	}
	// Acá tengo la referencia al objeto

Si hacemos una clase A asociada a una clase B , es porque en diseño estamos haciendo una asociación y en código estamos haciendo una referencia.

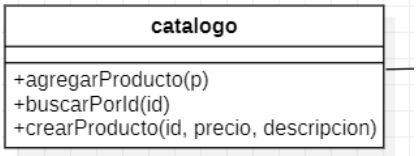
¿Cómo se obtiene la referencia?

1. **Se crea el objeto y se pasa la referencia**

Ejemplo de Larman,,, //
Producto p = new Producto(...); // creo el objeto p Producto
y luego cataloho lo agregar , pero no recibe el ID, si no el objeto
catalogo.agregarProducto(p); // catalogo recibe la referencia

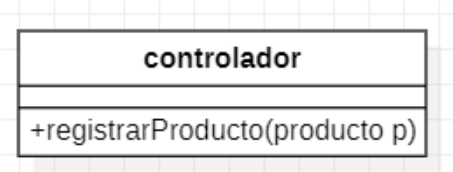


Tambien podes tener , catalogo.crearProducto(id, precio, descripcion); // Catalogo crea el objeto , usamos el patron GRASP CREADOR , entonces catalogo ya tiene la referencia



2. **Otro objeto ya tiene la referencia y la pasa, mediante algún metodo**

controlador.registrarProducto(producto); // controlador ya tiene la referencia y se la pasa a la clase que la necesite mediante un método



3. Se busca en una DB

Producto p = catalogo.buscarPorId(id) // usamos el ID… pero solo para **recuperar la referencia** después trabajás con el objeto

Como nos llego ese ID, puede ser mediante una interfaz, es decir un HTML que nos mando un ID ejemplo

```
<html lang="es">
<body>
<h2>Buscar producto por ID</h2>
<input type="number" id="idProducto" placeholder="Ingrese ID">
<button onclick="buscar()">Buscar</button>
<p id="resultado"></p>
<script>
```

En este caso, paso el id que recibí desde el frontend, pero no la referencia al objeto, luego busco el objeto, pero, quien lo busca??? depende del modelo que hice, pensemos que lo busca la clase catálogo, entonces seriaÑ

Producto p = catalogo.buscarPorId(idProducto);

En el gráfico de Larman:

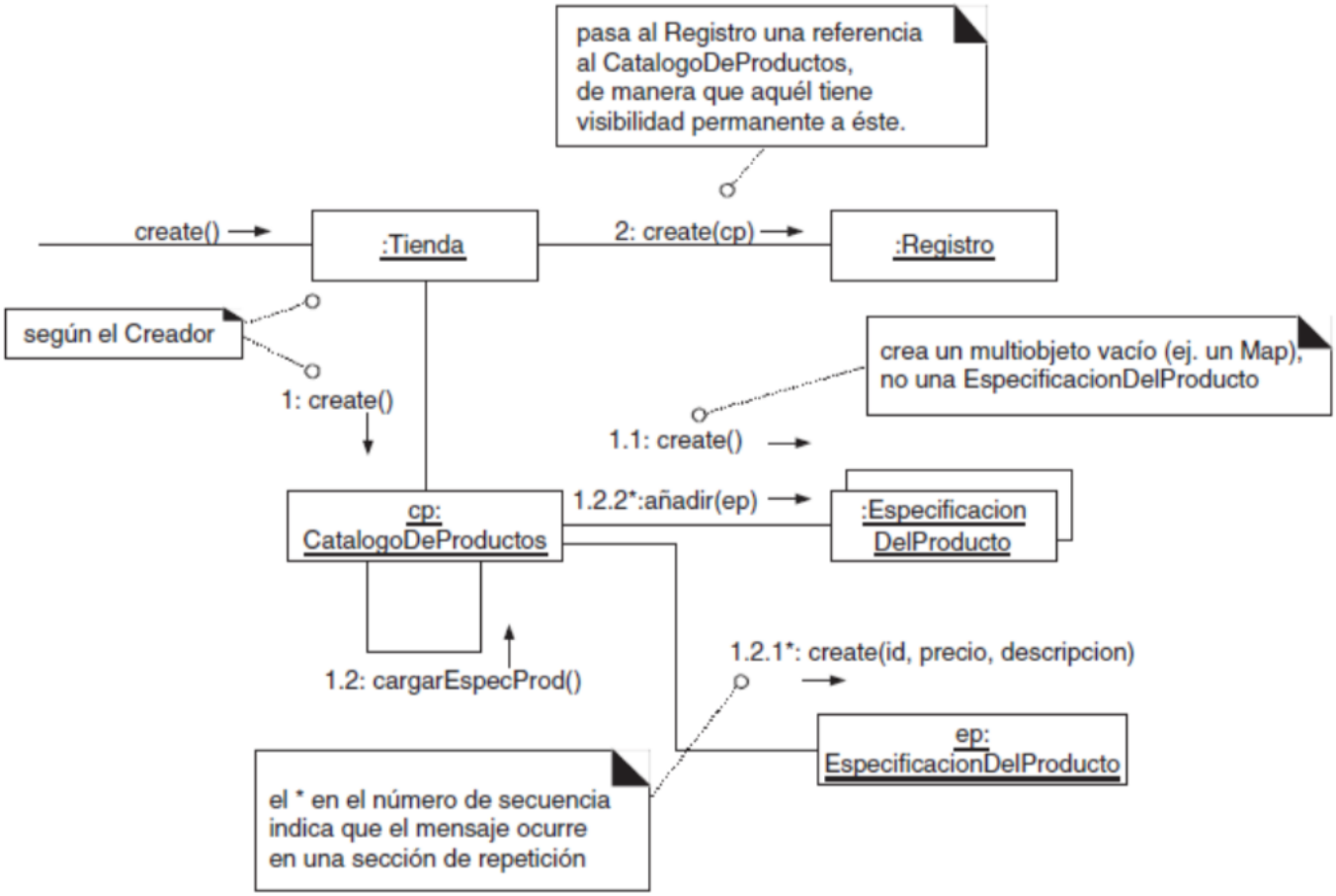


Figura 17.17. Creación del objeto del dominio inicial y los objetos siguientes.

- CatalogoDeProductos tiene una colección -CP// 1.2
- crea EspecificacionDelProducto -ep //1.2.1
- y luego hace: añadir(ep) , ahí guarda **la referencia ep**, no solo un ID //1.2.2

Si bien las asociaciones en diseño se representan como referencias, en algunos casos se utilizan IDs en lugar de referencias directas para reducir el acoplamiento entre clases, especialmente cuando intervienen BD o servicios externos o interfaz GUI.

Algo que no falla,, es pensar lo siguiente
UI → manda ID
Controlador → busca objeto con el ID
Dominio → trabaja con referencias, pero no siempre

DUDA

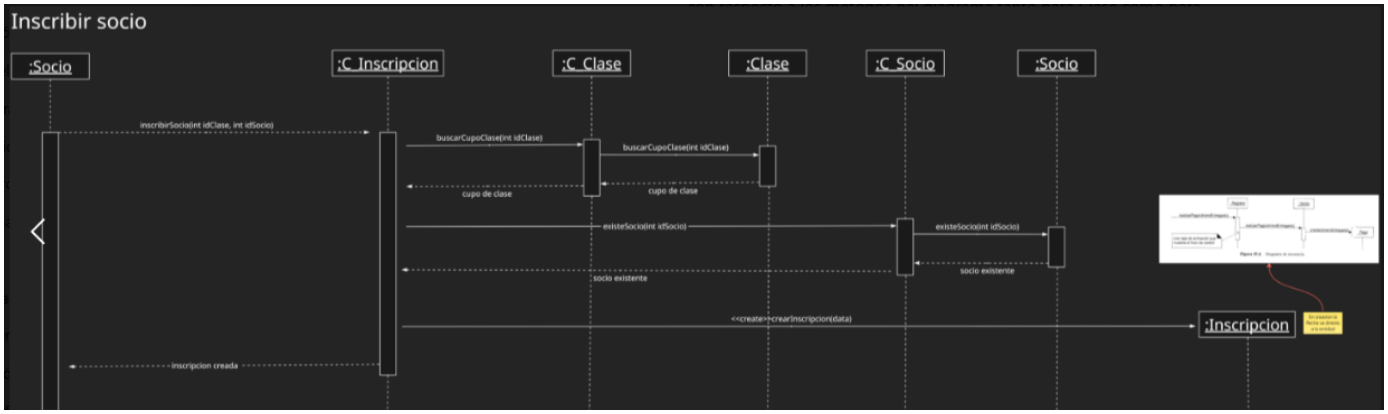
Estabamos haciendo el diagrama de secuencia y nos surgió una duda con respecto a los métodos del diagrama tanto para Clase como para Socio.

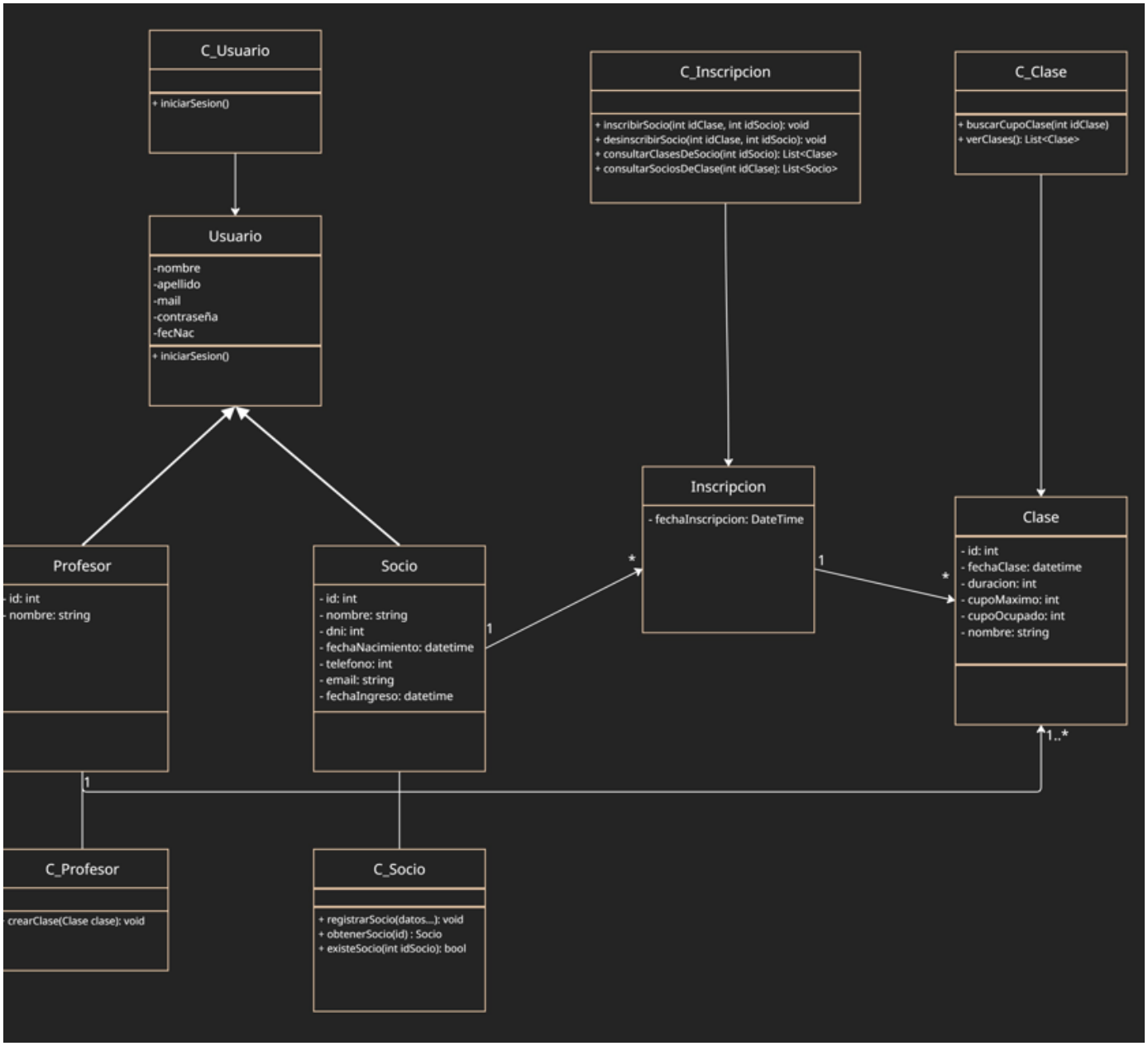
Por ejemplo en C_Inscripcion le pedimos a C_Clase si tiene cupo esa Clase con el método de buscarCupoClase C_Inscripcion → C_Clase → Clase De C_Inscripcion a C_Clase se utiliza el método de buscarCupoClase y el mismo método se usa de C_Clase a Clase, esto estaría bien? porque este método existe solo en C_Clase y no en Clase, pero cómo devolvemos que existe el cupo?

sería simplemente un getter del cupo y se lo trabaja en C_Clase y se devuelve a C_Inscripcion esto? Lo mismo nos pasa con Socio y si existe el socio.

En este caso es: C_Inscripcion → C_Socio → Socio

El método de existeSocio no existe en Socio y acá no tenemos ningún getter que nos indique si existe o no existe, qué le tendríamos que poner a la flechita?





Cada clase debe responder sobre su propio estado

Entonces:

- Clase sabe si tiene cupo
- C_Clase coordina / busca / delega
- C_Inscripcion orquesta el caso de uso

DUDA: Cuando quiero realizar el diagrama de secuencia siempre termino realizándolo después del de clases y siento, que me termina quedando mal por querer adaptarlo al diagrama que ya tengo hecho. ¿algún consejo para poder tomarlo como punto de partida para que me agregue valor?

1. Visibilidad de atributo

El objeto B es un campo permanente dentro de A.
Se usa cuando A siempre necesita a B para existir o trabajar.
Ejemplo:

```
class Profesor {
    private Catedra catedra;   // atributo: la cátedra que dicta

    public Profesor(Catedra catedra) {
        this.catedra = catedra;
    }
}
```

- Un Profesor siempre está asociado a una Cátedra.
- Conviene usar atributos cuando la relación es fuerte y estable.

2. Visibilidad de parámetro

El objeto B es pasado a un método de A para una operación puntual.
Se usa cuando B se necesita solo en un método, no todo el tiempo.
Ejemplo:

```
class Profesor {
    public void inscribirAlumno(Alumno alumno, Comision comision) {
        comision.agregarAlumno(alumno);
    }
}
```

- El Profesor recibe como parámetros un Alumno y una Comisión solo para ejecutar la inscripción.
- Conviene usar parámetros cuando el objeto no es parte fija del estado de la clase, sino de la operación.

3. Visibilidad local

El objeto B es creado o usado dentro de un método y desaparece cuando termina.
Se usa cuando el objeto es temporal, auxiliar o de uso interno.
Ejemplo:

```
class Profesor {
    public void tomarLista(Comision comision) {
        ListaAsistencia lista = new ListaAsistencia(); // variable local
        lista.generar(comision.getAlumnos());
    }
}
```

- La lista existe solo mientras se ejecuta tomarLista.
- Conviene usar variables locales cuando son objetos auxiliares que no deben quedar guardados en el estado de la clase.

4. Visibilidad global

El objeto B es accesible por muchos objetos sin necesidad de pasarlo.
Se suele implementar con Singleton o servicios estáticos.
Se recomienda con cuidado, porque genera acoplamiento fuerte.
Ejemplo:

```
class Sistema {
    public static RegistroDeAlumnos registroGlobal = new RegistroDeAlumnos();
}

class Alumno {
    public void inscribirse(Comision comision) {
        Sistema.registroGlobal.registrar(this, comision); // acceso global
    }
}
```

- Todos los objetos usan el mismo registro global de alumnos.
- Conviene usar global solo para recursos compartidos por todo el sistema (configuración, logs, conexión a base de datos).

Resumen:

- Atributo → relación permanente (Profesor–Cátedra).
- Parámetro → relación puntual en una operación (Profesor inscribe Alumno en Comisión).
- Local → objetos auxiliares (Lista de asistencia en un método).
- Global → recurso compartido en todo el sistema (Registro único de alumnos).

DUDA: que atributos debe tener un controller?

Según Larman, los Controller deben tener la menor cantidad posible de estado propio.
La idea principal es que el controller:

- coordine
- reciba eventos del sistema
- delegue trabajo a objetos del dominio

…pero que no concentre lógica de negocio ni datos que pertenecen a otras clases.

¿qué atributos puede tener un Controller?

Puede tener:
1. Referencias necesarias para coordinar, ejemplos:
    - objetos de sesión
    - repositories a datos
    - servicios
    - objetos del dominio que está creando u objetos del dominio usando temporalmente

2. Estado temporal del caso de uso/componente/modulo
Cuando el proceso de negocio dura varios pasos.
Ejemplo:
    - préstamoActual
    - socioSeleccionado
    - pedidoEnCurso

Lo que NO debe tener son Datos del dominio que pertenecen a otra clase
ejemplo:

```
class ControladorPrestamo {
   private Date fecha;
   private Cliente cliente;
   private List<Libro> libros; }
```

Porque esos datos deberían vivir en las clases que deben conocer esos datos, es decir en: préstamo, Cliente, Libro

Regla práctica
Preguntarse si.. “¿Este atributo representa información permanente del negocio?”
Si la respuesta es sí: probablemente NO debería estar en el controller.

DUDA: EJERCICIO GIMNASIO
Consulta profe: No quedó muy claro lo del gestor...me refiero a cuando es necesario usarlo y cuando no... entendí que es para bajar acoplamiento, pero pareciera que cómo en principios de diseño.....aparece siempre un gestor por controller.
Simplemente para pasarse la pelota (la responsabilidad) de atender una petición externa
A mí parecer debería aparecer solo cuando la cantidad de clases que maneja el controller es tanta que necesita dividir su gestión por medio de al menos 2 gestores
Ejemplo el controller de usuarios deriva el registro y el login a 2 gestores...Uno atiende los logins y otro los registros de distintos usuarios
Pero en el ejercicio de hacer el diagrama de secuencia de "inscripción a una clase" nos sugirió colocar un gestor y no entiendo por qué mejora la cohesión agregarlo si el controller de inscripciónes ("C_inscripcion") puede encargarse de eso por si mismo...
Ya que solo controla las clases: "clase", "Horario" e "Inscripción"
Para lo que si podría usarse un gestor es para la notificación (lo que sugirió que estaría bueno que se agregue en el caso del Gimnasio)

Sobre la consulta del Gestor, es importante no pensar que existe una regla del tipo “por cada Controller debe haber un Gestor”.
En Larman/GRASP, el Controller recibe una solicitud y debe coordinar el caso de uso, en nuestro caso hablamos de modulos o componentes, podriamos decir..conjuntos de casos de uso con una naturaleza en comun. A partir de ahí, puede delegar responsabilidades en otros objetos.

¿Cuándo tiene sentido agregar un Gestor? principalmente cuando necesitamos asignar una responsabilidad a una clase que permita mantener una buena cohesión y/o reducir el acoplamiento, especialmente cuando esa responsabilidad no corresponde a una entidad del dominio.

Por ejemplo, si el Controller comienza a tener muchas responsabilidades diferentes, puede ser conveniente delegar parte de esa lógica en un Gestor especializado.

Pero esto no significa que siempre que tengamos un Controller tengamos que agregar un Gestor. Si el Controller solamente coordina una operación sencilla y las responsabilidades están bien distribuidas, agregar un Gestor que simplemente “reciba el mensaje y se lo pase a otro objeto” no aporta una mejora real, como decís un pasapelotas.

En el caso de “Inscribir socio en una clase”, hay que analizar qué responsabilidades tiene realmente el Controller.

Si el Controller recibe la solicitud de inscripción y debe coordinar la búsqueda de la clase, verificar disponibilidad, crear la inscripción y relacionarla con el socio, podríamos tener un Controller que coordine esas operaciones sin necesidad de agregar un Gestor adicional. Sin embargo, si vemos que el Controller empieza a concentrar demasiada lógica del caso de uso, podemos utilizar un Gestor para encapsular esa lógica y mantener al Controller más enfocado en su responsabilidad de recibir y coordinar la solicitud, que es el objetivo principal, en este caso es controller es mas cohesivo.
El diseno es EVALUATIVO,, por eso, no usaría como criterio “si maneja más de X clases, entonces agrego un Gestor”. No existe ese número mágico. Hay que analizar las

responsabilidades y preguntarte, si ¿El Controller está quedando con demasiadas responsabilidades?(es decir esta muy saturado), si ¿La responsabilidad pertenece naturalmente a una entidad del dominio?, ¿si al agregar un Gestor realmente mejoras el diseño o simplemente agrega una clase que delega el mensaje?

Respecto de tu ejemplo de usuarios → gestor de login / gestor de registro, sí podría tener sentido si esas operaciones tienen responsabilidades y reglas suficientemente diferentes como para justificar su separación.

En cambio, si tenes ---Controller → Gestor → Clase
y el Gestor simplemente recibe el mensaje y se lo pasa a la clase, sin aportar ninguna responsabilidad propia, probablemente no estamos obteniendo una mejora de diseño, pero si estas pensando en agregar mas funcionalidad, entonces al atomizarlo hasta ese nivel estas agregrando el atributo de calidad EXTENSIBLE, en algun momento vas a darle mayor nivel de detalle y funcionalidad, entonces en este caso no te puedo decir esta bien o esta mal..

Finalmente, respecto de la notificación, ahí sí puede aparecer claramente un objeto especializado, porque enviar una notificación es una responsabilidad que no necesariamente corresponde a las entidades del dominio (Socio, Clase, Inscripción, etc.).
Podes utilizar una clase como GestorNotificaciones o similar para encapsular esa responsabilidad y evitar que el Controller o las entidades tengan que conocer los detalles de cómo se realiza la notificación, como arma el mensaje , como es la vía de comunicación etc.

En resumen: el Gestor no se agrega por obligación ni “porque hay un Controller”. Se agrega cuando ayuda a distribuir responsabilidades de manera que el diseño tenga mejor cohesión y menor acoplamiento. un controller suele tener varios gestores

En tus dudas pusiste “debería aparecer solo cuando la cantidad de clases que maneja el Controller es tanta que necesita dividir su gestión”., esto es bastante intuitivo y te puede ayudar, pero no es exactamente el criterio del autor en que nos basamos, Larman dice que no importa cuántas clases maneja, sino qué responsabilidades tiene.

Por ejemplo:
C_Inscripcion/Gestor → Clase
C_Inscripcion/Gestor → Horario
C_Inscripcion/Gestor → Inscripcion

Un Controller/ Gestor puede perfectamente coordinar varias clases (participantes) de un mismo caso de uso.

El problema sería que `C_Inscripcion` además empiece a:
* gestionar pagos,
* enviar emails,
* registrar auditoría,
* generar reportes,
* dar de altas ejercicios… Eso es lo que ponemos FOCO